BÁO CÁO LẦN 1

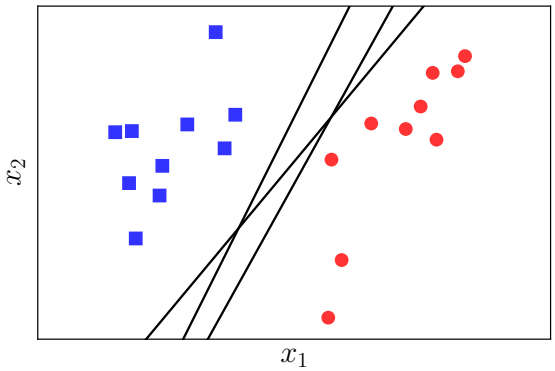
Người thực hiện: Nguyễn Mạnh Thắng - 20235426

I Giới thiệu.

Giả sử, ta có một tập dữ liệu có 2 nhãn $(1, -1)$, đã biết trước các đặc trưng của các điểm dữ liệu, ý tưởng cơ bản của thuật toán SVM sẽ là đi tìm một mặt siêu mặt phẳng: $\mathbf{a}^T\mathbf{x} + b = 0$, để phân biệt tập dữ liệu thành 2 phần riêng biệt, một nửa sẽ chỉ chứa các điểm có nhãn 1, nửa còn lại sẽ chứa các điểm có nhãn -1. Tất nhiên, với ý tưởng cơ bản như thế này, thuật toán SVM sẽ chỉ có thể làm việc được với tập dữ liệu *linearly seperable*, đối với các tập dữ liệu mà không thể *linearly seperable* ta sẽ có các biến thể khác của SVM mà cho hiệu suất phân loại tốt hơn: Soft Margin SVM, Kernel SVM. Do đang hoàn thiện nốt phần kiến thức tối ưu cần thiết(Duality), nên ở báo cáo này tôi sẽ trình bày về phiên bản cơ bản nhất của SVM, và tập dữ liệu dùng để minh hoạ cũng sẽ *linearly seperable*.

II Mô hình toán học.

Ý tưởng cụ thể.

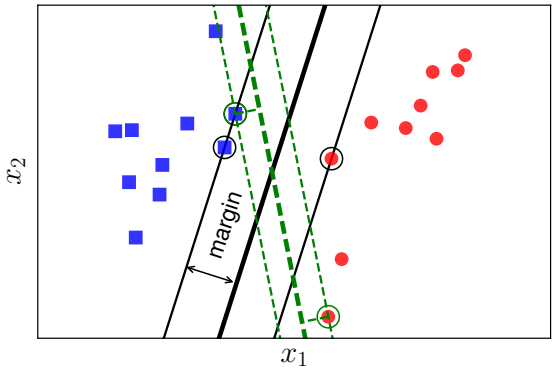


Hình 1: Các mặt phân cách hai classes *linearly separable*.

Chúng ta nhận thấy rằng, với tư tưởng ban đầu là tìm một *hyperplane* để tách tập dữ liệu thành 2 vùng riêng biệt, mỗi vùng chỉ tồn tại một *class* duy nhất thì có thể tồn tại nhiều hơn một *hyperplane*, như ví dụ trong Hình 1. Vậy bây giờ chúng ta sẽ nên chọn *hyperplane* dự theo tiêu chuẩn nào để có thể thu được kết quả phân loại tốt?

Ta đưa ra **quan sát** đầu tiên: dựa trên Hình 1 ta thấy có 2 *hyperplane* ở vị trí gần *class* đỏ(tròn) hơn so với *class* xanh(vuông). Nếu như ta chọn một trong 2 *hyperplane* này làm *hyperplane* phân loại thì có thể xảy ra trường hợp: một điểm được phân loại là xanh, nhưng mà điểm đó lại ở gần với *hyperplane* phân loại, mà *hyperplane* phân loại lại ở gần các điểm đỏ nên một cách định tính ta có thể thấy được điểm này đáng lẽ sẽ phải được phân loại là đỏ. Ta rút ra kết luận *hyperplane* phân loại sẽ không được ở quá gần bất kỳ một tập điểm nào, đỏ hoặc xanh, hay để dễ dàng cho việc tối ưu về sau *hyperplane* phân loại phải cách đều hai tập điểm.

Chú thích: Khoảng cách từ một tập điểm đến hyperplane được định nghĩa là khoảng cách từ điểm gần nhất của tập điểm đó đến hyperplane.



Hình 2: Các mặt phân cách cách đều hai *classes linearly separable*.

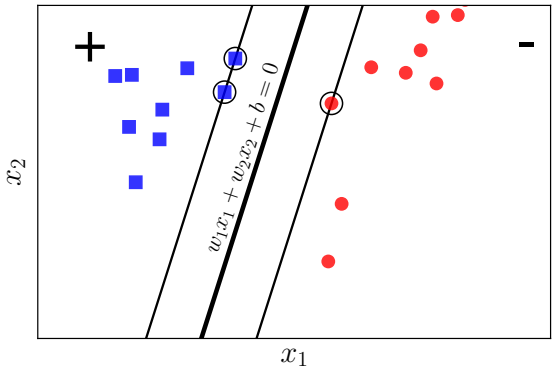
Tiếp tục, ta đưa ra **quan sát** thứ hai: nhìn vào Hình 2 ta thấy lại có thể tồn tại nhiều *hyperplane* mà có thể cách đều hai tập điểm, đỏ và xanh, vậy bây giờ ta sẽ chọn *hyperplane* nào? Bổ sung thêm vào kết luận được rút ra từ **quan sát** đầu tiên, ta thấy rằng mặc dù *hyperplane* phân loại đã cách đều 2 tập điểm, đỏ và xanh, nhưng nếu như khoảng cách vẫn nhỏ thì vẫn có thể xảy ra vấn đề như đã trình bày ở **quan sát** đầu tiên. Vậy nên để có thể tối ưu việc phân loại thì ta sẽ chọn *hyperplane* có khoảng cách lớn nhất. Đây chính là ý tưởng chính của thuật toán SVM, hay còn gọi là Maximum Margin Classifier, đó là tìm *hyperplane* phân loại cách đều 2 tập điểm(đỏ và xanh) với một khoảng cách lớn nhất.
Chú thích: Khoảng cách gần nhất từ một điểm đến một hyperplane sẽ được gọi là margin.

Hình thành mô hình toán học.

Giả sử rằng các cặp dữ liệu của *training set* là

$$(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)$$

với vector $\mathbf{x}_i \in \mathbb{R}^d$ thể hiện đầu vào của một điểm dữ liệu và y_i là nhãn của điểm dữ liệu đó. d là số chiều của dữ liệu và N là số điểm dữ liệu. Giả sử rằng nhãn của mỗi điểm dữ liệu được xác định bởi $y_i = 1$ (class xanh) hoặc $y_i = -1$ (class đỏ), tức là các điểm màu xanh sẽ ở phía dương của *hyperplane*, các điểm màu đỏ sẽ nằm ở phía âm (đây là giả sử quan trọng để có thể giải được thuật toán SVM, sẽ có giải thích ở phía dưới).



Hình 3: Phân tích bài toán SVM.

Ta quan sát thấy một điểm quan trọng sau đây: với cặp dữ liệu $(\mathbf{x}_n, y_n)$ bất kỳ, khoảng cách từ điểm đó tới *hyperplane* phân loại là:

$$\frac{y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)}{\|\mathbf{w}\|_2}$$

Điều này có thể dễ nhận thấy vì theo giả sử ở trên, y_n luôn cùng dấu với phía của $\mathbf{x}_n$. Từ đó suy ra y_n cùng dấu với $(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)$, và tử số luôn là một số không âm.

Vì *margin* được tính là khoảng cách gần nhất từ 1 điểm tới mặt đó (bất kể điểm nào trong hai *classes*):

$$\text{margin} = \min_n \frac{y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \tag{1}$$

Bài toán tối ưu trong SVM chính là bài toán tìm $\mathbf{w}$ và b sao cho *margin* này đạt giá trị lớn nhất:

$$\begin{aligned} (\mathbf{w}, b) &= \arg\max_{\mathbf{w},b} \left\{ \min_n \frac{y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b)}{\|\mathbf{w}\|_2} \right\} = \arg\max_{\mathbf{w},b} \left\{ \frac{1}{\|\mathbf{w}\|_2} \min_n y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) \right\} \\ \text{vđk:} \quad & y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) > 0, \quad \forall n = 1, 2, \ldots, N \end{aligned} \tag{2}$$

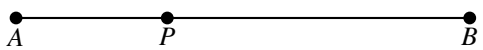
Nhìn vào phương trình (2), ta chú ý thấy điều kiện $y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) > 0, \quad \forall n = 1, 2, \ldots, N$, điều kiện này giúp giới hạn không gian tìm kiếm để đảm bảo sao cho các giá trị $\mathbf{w}, b$ thoả mãn luôn cho ta một mặt phẳng phân chia rời rạc 2 tập điểm, tức là điểm xanh sẽ ở phía dương, điểm đỏ sẽ ở phía âm. Ví dụ cụ thể:

$$\begin{aligned} y_i = 1(\text{điểm xanh}) &\Rightarrow (\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) > 0 \\ y_i = -1(\text{điểm đỏ}) &\Rightarrow (\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) < 0 \end{aligned}$$

như vậy, khi hợp các điều kiện này lại với nhau ta sẽ thu được không gian các giá trị $\mathbf{w}, b$ thoả mãn. Đây chính là lý do tại sao giả sử phía trên lại quan trọng.

Để ý một chút, ta có thể thấy, ý tưởng của thuật toán SVM ta vừa nêu ở trên là: "*tìm hyperplane phân loại cách đều 2 tập điểm(đỏ và xanh) với một khoảng cách lớn nhất*", nhưng ở bài toán tối ưu trong SVM (2) thì lại đi *maximum* giá trị của *margin*. Một cách trực quan, ta chưa thấy rõ được sự liên kết giữa bài toán tối ưu (2) với ý tưởng ban đầu(mục tiêu ban đầu), nhưng thực ra bài toán tối ưu (2) với mục tiêu ban đầu lại hoàn toàn giống nhau. Điều này sẽ được thể hiện qua một chứng minh phản chứng, sau đây.

Chứng minh:
+) Giả định, ta đã giải quyết được bài toán tối ưu (2).
+) Gọi $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$ là một *hyperplane* cho kết quả tối ưu; d_1, d_2 lần lượt là khoảng cách gần nhất từ tập xanh, tập đỏ đến $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$.
+) Giả sử: $d_1 \neq d_2$, không mất tính tổng quát ta có thể coi $d_1 < d_2 \Rightarrow \text{margin} = d_1$.
+) Minh hoạ d_1, d_2 trên trên hình vẽ, với $AP = d_1, BP = d_2$.



+) Nhìn vào hình vẽ, ta thấy ta hoàn toàn có thể thu được giá trị lớn hơn của d_1 bằng cách dịch điểm P về phía chính giữa đoạn thẳng, hay tức là ta có thể thu được giá trị margin lớn hơn bằng cách dịch mặt $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$ về phía tập đỏ. Nhưng mà mặt $\mathbf{m}_1^{\mathrm{T}}\mathbf{x} + m_2 = 0$ đã là một phương án tối ưu rồi, vậy nên không thể tồn tại bất kỳ mặt nào khác có thể cho giá trị margin lớn hơn nữa.
+) Từ đó, ta kết luận rằng giả sử $d_1 \neq d_2$ là vô lý, không thể xảy ra, hay là khi tìm được kết quả tối ưu của bài toán (2) thì d_1 luôn bằng d_2.
+) Từ kết luận trên, ta đưa ra nhận xét: khi giải bài toán tối ưu (2) chính là việc mình đang đi tìm các *hyperplane* cách đều 2 tập với một khoảng cách lớn nhất, tức là hoàn toán giống với mục tiêu của thuật toán SVM $\rightarrow$ đpcm.

Một nhận xét quan trọng nhất của bài toán tối ưu (2): nếu ta thay vector hệ số $\mathbf{w}$ bởi $k\mathbf{w}$ và b bởi kb trong đó k là một hằng số dương thì khoảng cách từ từng điểm đến *hyperplane* phân loại không đổi, tức *margin* không đổi. Dựa trên tính chất này, ta có thể giả sử:

$$y_n\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b\right) = 1$$

với những điểm gần *hyperplane* phân loại nhất.

Như vậy, bài toán tối ưu (2) sẽ trở thành:

$$
\begin{aligned}
(\mathbf{w}, b) &= \arg\max_{\mathbf{w},b} \left\{ \frac{1}{\|\mathbf{w}\|_2} \right\} \\
\text{vđk:} \quad & y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) \geq 1, \quad \forall n = 1, 2, \ldots, N
\end{aligned}
\tag{3}
$$

Biến đổi đơn giản một chút, nó sẽ trở thành:

$$
\begin{aligned}
(\mathbf{w}, b) &= \arg\min_{\mathbf{w},b} \left\{ \|\mathbf{w}\|_2^2 \right\} = \mathbf{w}^{\mathrm{T}}\mathbf{I}\mathbf{w} \\
\text{vđk:} \quad & 1 - y_n(\mathbf{w}^{\mathrm{T}}\mathbf{x}_n + b) \leq 0, \quad \forall n = 1, 2, \ldots, N
\end{aligned}
\tag{4}
$$

Nhìn vào bài toán tối ưu (4), ta thấy hàm mục tiêu có *Hessian* là $\mathbf{I}$, xác định dương, nên đây là hàm lồi chặt, các hàm *constraint* là các hàm tuyến tính nên đây cũng là các hàm lồi, do đó bài toán tối ưu (4) là bài toán tối ưu hàm lồi, hay cụ thể là *Quadratic Programming*. Đến đây thì ta có thể sử dụng thư viên tối ưu lồi *CVXOPT* trong *Python* để giải.

III Dataset.

Link dataset trên Kaggle. Đây là link đến bộ dataset trên Kaggle, tập dữ liệu có dữ liệu 5172 email đã được phân loại *spam or not spam*. Các email không phải ở dạng thuần text mà mỗi email đã được biểu diễn bởi một vector có 3000 *features*, mỗi *feature* là một từ, giá trị của các *feature* là số lần xuất hiện của từ đó trong email.